



Spring Remote

1

Spring Remote

2

- ✓ Introduction
- ✓ RMI
- ✓ Hessian / Burlap / HTTP Invoker
- ✓ JAX-WS

Présentation

- Spring remote a pour objectif de vous permettre d'exposer et récupérer des services distants.
- Ils peuvent être réalisés en :
 - JAX-WS : Web Service XML classique en SOAP
 - JAX-RPC : ancêtre du JAX-WS
 - Remote Method Invocation : RMI.
 - Spring HTTP invoker : api spécifique Java/Spring
 - Hessian ou Burlap : API spécifique Caucho.
 - JMS : Java Messaging service
 - AMQP : Advanced Message Queuing Protocol pour Spring >= 4

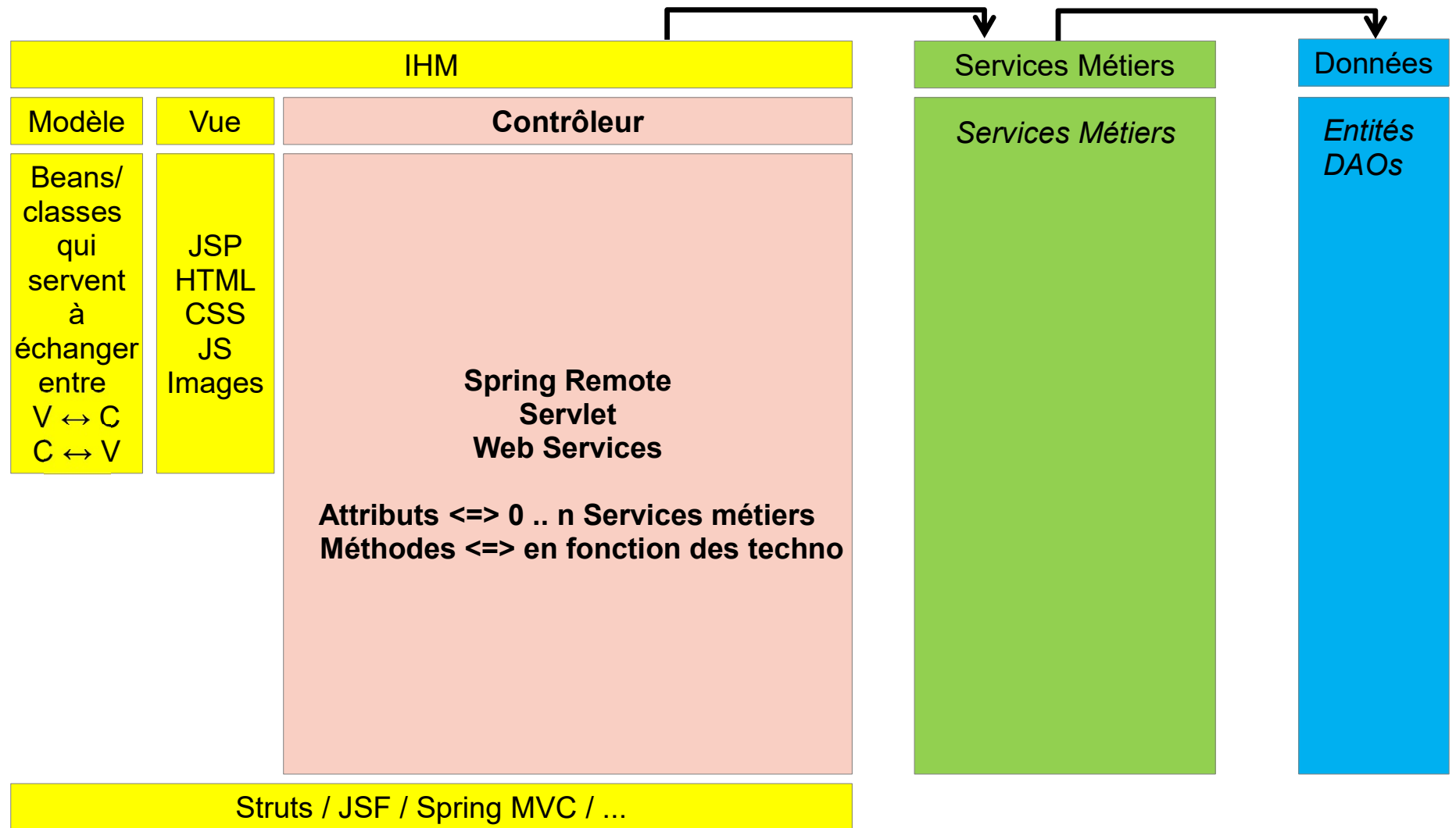
Présentation

- Chaque implémentation a ses avantages et ses inconvénients
 - RMI : Echange en binaire par sérialisation Java. Interopérable. Le plus rapide de tous.
 - Spring HTTP invoker : Echange en binaire par sérialisation Java. Seulement Java.
 - Hessian : Echange en binaire par sérialisation. Interopérable.
 - Burlap : Echange en XML.
 - JMS : Interopérable.
 - AMQP : Interopérable.

Présentation

- Vous noterez qu'il n'y a pas les web services REST dans cette liste
- Spring remote peut servir dans le cas d'architecture en micro service
 - Mais sans REST
- Cependant, il existe un module qui fait '*comme*' spring-remote pour les services REST :
 - <https://github.com/ggeorgovassilis/spring-rest-invoker>
- Attention : même si on réalise des services distants se sont des contrôleurs (⇔ @Controller)
 - Et pas des services métiers (⇔ @Service)

Présentation



Spring Remote : RMI

- Vous permet de réaliser des services RMI classiques
- Vous devrez donc réaliser :
 - Une interface pour chaque service distant
 - ❑ Elle doit hériter de `java.rmi.Remote`
 - ❑ Ses méthodes doivent lever des `java.rmi.RemoteException`
 - ❑ Ses paramètres / éléments retournés doivent être sérialisables
 - Une classe concrète par interface
 - ❑ Elle sera surement liées à vos services métiers via des `@Autowired`
- La RMI Registry sera gérée par Spring lors de la déclaration de chaque services

Spring Remote : RMI – côté serveur

➤ Vos services distants seront :

➡ Annotés via `org.springframework.stereotype.Controller`

❑ N'oubliez pas votre component-scan

➡ Montés via un fichier XML de configuration Spring

```
<bean class="org.springframework.remoting.rmi.RmiServiceExporter">
  <property name="registryPort" value="1199" />
  <property name="serviceName" value="AuthentificationRemoteService" />
  <property name="service" ref="authentificationRemoteService" />
  <property name="serviceInterface" value="com.banque.service.remote.IAuthentificationRemoteService" />
  <!-- Notre service sera disponible sur l'url : rmi://localhost:1199/AuthentificationRemoteService -->
</bean>
```


Spring Remote : RMI – côté client

- Vous déclarez simplement vos beans distants

```
<bean id="clientAuthenticationRemoteService" class="org.springframework.remoting.rmi.RmiProxyFactoryBean">  
  <property name="serviceUrl" value="rmi://localhost:1199/AuthenticationRemoteService" />  
  <property name="serviceInterface" value="com.banque.service.remote.IAuthenticationRemoteService" />  
</bean>
```

- Remarque : l'interface Java qui représente vos beans devra être déployée côté client

Travaux pratiques

- Exo 21 - Réalisé le premier TP Spring remote
 - En repartant des exercices où les DAO, entités, services métiers sont codés en Spring, ajoutez une couche de contrôleur en Spring Remote RMI



Spring Remote : Hessian / Burlap / HttpInvoker

- Vous permet de réaliser des services distants sans les *contraintes* RMI
- Vous êtes dans une approche J2EE ⇔ il faudra un serveur d'application web
 - C'est la servlet DispatcherServlet du Spring qui s'occupera du routage
- Les trois sont des implémentations spécifiques, les informations seront échangées sur de l'HTTP en binaire ou en XML selon les implémentations

Spring Remote : Hessian / Burlap / HttpInvoker

- Dans les trois cas, vous devrez :
 - Déclarer la servlet Spring dans votre fichier web.xml
 - ❑ Faire le fichier de configuration Spring qui lui sera associé
 - Réaliser une interface pour chaque service distant
 - ❑ Vous pouvez lever les exceptions que vous voulez
 - ❑ Ses paramètres / éléments retournés doivent être sérialisables
 - Réaliser une classe concrète par interface
 - ❑ Elle sera sûrement liées à vos services métiers via des @Autowired

Spring Remote : Hessian / Burlap / HttpInvoker

- Dans cette approche, l'implémentation (Hessian, Burlap, ...) ne transparait pas du tout dans le code.
- On ne choisit son implémentation qu'au moment de déclarer ses services distants
 - Côté serveur : Généralement dans le fichier de configuration Spring associé à la DispatcherServlet
 - Côté client : dans le fichier Spring dédié au client

Spring Remote : Hessian / Burlap / HttpInvoker

14

➤ Côté serveur (fichier de configuration Spring de la servlet) :

```
<bean name="/AuthenticationRemoteService" class="org.springframework.remoting.caucho.HessianServiceExporter">
  <!-- Notre bean est monte par annotation -->
  <property name="service" ref="authenticationRemoteService" />
  <property name="serviceInterface" value="com.banque.service.remote.IAuthenticationRemoteService" />
  <!-- Notre service sera disponible sur l'url : http://localhost:8080/exo18.spring.remote.hessian/remoting/AuthenticationRemoteService -->
</bean>
```

```
<bean name="/AuthenticationRemoteService" class="org.springframework.remoting.httpinvoker.HttpInvokerServiceExporter">
  <!-- Notre bean est monte par annotation -->
  <property name="service" ref="authenticationRemoteService" />
  <property name="serviceInterface" value="com.banque.service.remote.IAuthenticationRemoteService" />
  <!-- Notre service sera disponible sur l'url : http://localhost:8080/exo18.spring.remote.httpinvoker/remoting/AuthenticationRemoteService -->
</bean>
```

- C'est l'implémentation de la classe qui change
- Notez que l'URL final est dépendant :
 - ➡ Du context root de votre application J2EE
 - ➡ Du mapping de votre DispatcherServlet

Spring Remote : Hessian / Burlap / HttpInvoker

15

➤ Côté client :

```
<bean id="clientAuthenticationRemoteService" class="org.springframework.remoting.caucho.HessianProxyFactoryBean">
  <property name="serviceUrl" value="http://localhost:8080/exo18.spring.remote.hessian/remoting/AuthenticationRemoteService" />
  <property name="serviceInterface" value="com.banque.service.remote.IAuthenticationRemoteService" />
</bean>
```

```
<bean id="clientAuthenticationRemoteService" class="org.springframework.remoting.httpinvoker.HttpInvokerProxyFactoryBean">
  <property name="serviceUrl" value="http://localhost:8080/exo18.spring.remote.httpinvoker/remoting/AuthenticationRemoteService" />
  <property name="serviceInterface" value="com.banque.service.remote.IAuthenticationRemoteService" />
</bean>
```

- Ici aussi, seule l'implémentation de la classe change
- Tout comme pour le RMI, les interfaces Java qui représentent vos services distants devront être déployées côté client

Travaux pratiques

- Exo 21 - En repartant des exercices où les DAO, entités, services métiers sont codés en Spring, ajoutez une couche de contrôleur en Spring Remote Hessian, puis http Invoker



Spring Remote : JAX-WS

- Vous permet de vous connecter simplement à des web services distants de type SOAP
- Deux approches possibles :
 - Sans serveur J2EE
 - Approche J2EE ⇔ il faudra un serveur d'application web
- La réalisation du service SOAP sera classique (respectera la norme Java JAX-WS)
- Tous se passera lors de la configurations Spring.

Spring Remote : JAX-WS – P1

18

- La première approche fera usage de l'objet *SimpleJaxWsServiceExporter* du Spring
- Il n'y a pas de serveur J2EE dans ce cas
- Cette approche est pratique si vous n'avez **qu'un seul** web service
 - Une seule classe de web service

Spring Remote : JAX-WS - P1

➤ Côté 'serveur' (pas un vrai serveur) :

```
<bean class="org.springframework.remoting.jaxws.SimpleJaxWsServiceExporter">  
  <!-- NE PAS OUBLIER LE '/' a la fin -->  
  <property name="baseAddress" value="http://localhost:8080/" />  
</bean>
```

- Il n'y a rien d'autre à faire, votre service Web doit être correctement déclaré
 - Interface en @WebService + méthodes annotées
 - Classe concrètes en @Controller et @WebService

Spring Remote : JAX-WS – P2

- Dans la seconde solution, nous devons déclarer une servlet dans le fichier web.xml
 - Attention : ce n'est pas le DispatcherServlet
- Elle portera les déclarations/mapping de nos services webs
- Nous sommes en serveur J2EE

Spring Remote : JAX-WS – P2

21

➤ Déclaration dans le fichier web.xml

```
19<= <servlet>
20   <servlet-name>WSSpringServlet</servlet-name>
21   <servlet-class>com.sun.xml.ws.transport.http.servlet.WSSpringServlet</servlet-class>
22 </servlet>
23<= <servlet-mapping>
24   <servlet-name>WSSpringServlet</servlet-name>
25   <!-- Chaque URL correspond a un url de wss:binding -->
26   <url-pattern>/AuthenticationRemoteService</url-pattern>
27   <url-pattern>/CompteRemoteService</url-pattern>
28   <url-pattern>/OperationRemoteService</url-pattern>
29 </servlet-mapping>
30<= <!-- Vous pouvez aussi declarer n servlet
31   WSSpringServlet sur un URL dedie-->
```

➤ Nous avons ici, 3 web services ⇔ 3 classes distinctes

Spring Remote : JAX-WS – P2

- Côté serveur, en plus de la déclaration de la servlet, il faudra réaliser un fichier Spring qui précisera la déclaration de vos web services
- Notez l'utilisation d'un nouveau name space

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xmlns:context="http://www.springframework.org/schema/context"
5       xmlns:wss="http://jax-ws.dev.java.net/spring/servlet"
6       xmlns:ws="http://jax-ws.dev.java.net/spring/core"
7       xsi:schemaLocation="http://www.springframework.org/schema/beans
8         http://www.springframework.org/schema/beans/spring-beans.xsd
9         http://www.springframework.org/schema/context
10        http://www.springframework.org/schema/context/spring-context.xsd
11        http://jax-ws.dev.java.net/spring/core http://jax-ws.java.net/spring/core.xsd
12        http://jax-ws.dev.java.net/spring/servlet http://jax-ws.java.net/spring/servlet.xsd">
13
14   <context:component-scan base-package="com.banque.service.remote" />
15
16   <wss:binding url="/AuthenticationRemoteService">
17     <wss:service>
18       <ws:service bean="#authenticationRemoteServiceEndpoint" />
19     </wss:service>
20   </wss:binding>
21
22   <wss:binding url="/CompteRemoteService">
23     <wss:service>
24       <ws:service bean="#compteRemoteServiceEndpoint" />
25     </wss:service>
26   </wss:binding>
27
28   <wss:binding url="/OperationRemoteService">
29     <wss:service>
30       <ws:service bean="#operationRemoteServiceEndpoint" />
31     </wss:service>
32   </wss:binding>
33 </beans>
```

ID du bean
Spring ⇔ ref

Spring Remote : JAX-WS

➤ Côté client, dans les deux cas :

```
<bean id="authenticationWebService" class="org.springframework.remoting.jaxws.JaxWsPortProxyFactoryBean">
  <property name="serviceInterface" value="com.banque.service.remote.IAuthenticationRemoteService" />
  <property name="wsdlDocumentUrl" value="http://localhost:8080/exo18.spring.remote.jaxws2/AuthenticationRemoteService?WSDL" />
  <!-- nomDeLaClasse + "Service" -->
  <property name="serviceName" value="AuthenticationRemoteServiceEndpointService" />
  <!-- nomDeLaClasse + "Port" -->
  <property name="portName" value="AuthenticationRemoteServiceEndpointPort" />
  <!-- Nom du package a l'envers -->
  <property name="namespaceUri" value="http://impl.remote.service.banque.com/" />
</bean>
```

➤ Tout comme pour les autres modes, les interfaces Java qui représentent vos services distants devront être déployées côté client

Spring Remote : JAX-WS

- Côté code, dans les deux cas votre interface doit être correctement annotée :

```
1 package com.banque.service.remote;
2
3 import javax.jws.WebMethod;
4 import javax.jws.WebParam;
5 import javax.jws.WebResult;
6 import javax.jws.WebService;
7 import javax.jws.soap.SOAPBinding;
8
9 import com.banque.entity.impl.UtilisateurEntity;
10
11 * Service distant d'authentification. <br/>
12 @WebService(serviceName = "AuthenticationRemoteService")
13 @SOAPBinding(style = SOAPBinding.Style.DOCUMENT, use = SOAPBinding.Use.LITERAL)
14 public interface IAuthenticationRemoteService {
15
16     * Authentifie un utilisateur.
17     @WebMethod(operationName = "authentifier", action = "urn:authentifier")
18     @WebResult(name = "user")
19     public abstract UtilisateurEntity authentifier(@WebParam(name = "pLogin") String pLogin,
20         @WebParam(name = "pPassword") String pPassword) throws Exception;
21 }
22
```

- Il n'y a pas d'annotation Spring dans l'interface

Spring Remote : JAX-WS

- Côté code, dans les deux cas votre classe doit être correctement annotée :
- Les deux seules annotations Spring n'ont pas de rapport avec JAX

```
1 package com.banque.service.remote.impl;
2
3 import javax.jws.WebService;
4
5 import org.apache.log4j.LogManager;
6 import org.apache.log4j.Logger;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.stereotype.Controller;
9
10 import com.banque.entity.impl.UtilisateurEntity;
11 import com.banque.service.IAuthenticationService;
12 import com.banque.service.remote.IAuthenticationRemoteService;
13
14 * Gestion de l'authentification distante JAX-WS. <br/>
15 @Controller
16 @WebService(endpointInterface = "com.banque.service.remote.IAuthenticationRemoteService")
17 public class AuthenticationRemoteServiceEndpoint extends AbstractRemoteService
18     implements IAuthenticationRemoteService {
19     private static final Logger LOG = LogManager.getLogger(AuthenticationRemoteServiceEndpoint.class);
20     @Autowired
21     private IAuthenticationService service;
22
23     * Recupere le service associe.
24     protected IAuthenticationService getService() {}
25
26     public UtilisateurEntity authentifier(String pLogin, String pPassword) throws Exception {}
27 }
28 }
```

Spring Remote : JAX-WS

- Côté code, JAX-WS supporte mal les interfaces et le polymorphisme
- Il est préférable d'éviter d'avoir des interface sur ses entités (ou DTO)
 - Sur les objets en entrées ou sorties de vos web services
- JAX-WS fera usage de JAX-B, c'est à vous de réaliser le binding XML sur vos entités (ou DTO)

Travaux pratiques

- Exo 21 - En repartant des exercices où les DAO, entités, services métiers sont codés en Spring, ajoutez une couche de contrôleur en Spring Remote JAX-WS via la méthode 1 puis via la méthode 2

